

TransitOps — Full Build Plan (2 people, 8 hours)

Stack: React (frontend) + Node/Express (backend) + PostgreSQL (DB) + Claude/OpenAI API (wow features)

1. Data Model (PostgreSQL)

```
-- USERS & ROLES
users (id, name, email, password_hash, role
ENUM('fleet_manager','driver','safety_officer','financial_analyst'), created_at)

-- VEHICLES
vehicles (
  id, registration_number UNIQUE, name_model, type,
  max_load_capacity, odometer, acquisition_cost,
  status ENUM('Available','On Trip','In Shop','Retired'),
  region, created_at
)

-- DRIVERS
drivers (
  id, name, license_number, license_category, license_expiry_date,
  contact_number, safety_score, status ENUM('Available','On Trip','Off
Duty','Suspended'),
  created_at
)

-- TRIPS
trips (
  id, source, destination, vehicle_id FK, driver_id FK,
  cargo_weight, planned_distance, actual_distance, fuel_consumed,
  status ENUM('Draft','Dispatched','Completed','Cancelled'),
  dispatched_at, completed_at, created_at
)

-- MAINTENANCE LOGS
maintenance_logs (
  id, vehicle_id FK, issue_description, cost,
  status ENUM('Active','Closed'), created_at, closed_at
)

-- FUEL LOGS
fuel_logs (id, vehicle_id FK, trip_id FK NULLABLE, liters, cost, date)

-- EXPENSES
expenses (id, vehicle_id FK, type ENUM('Toll','Other'), amount, date,
description)
```

Relationships: Vehicle 1—N Trips, Driver 1—N Trips, Vehicle 1—N Maintenance, Vehicle 1—N Fuel Logs/Expenses.

2. Business Rules → Where They Live (backend, never trust frontend alone)

Rule	Implementation point
Registration number unique	DB constraint + API validation
Retired/In Shop hidden from dispatch	Vehicle selection query filters <code>status = 'Available'</code>
Expired license / Suspended driver blocked	Trip creation validation: check <code>license_expiry_date > NOW()</code> and <code>status = 'Available'</code>
Driver/vehicle already On Trip blocked	Same validation layer
Cargo weight $\leq$ max load capacity	Validation on trip create/dispatch
Dispatch → both go On Trip	Transaction: update trip status + vehicle status + driver status atomically
Complete → both go Available	Same atomic transaction pattern
Cancel dispatched trip → restore Available	Same pattern
Active maintenance → vehicle In Shop	On maintenance log create, trigger vehicle status update
Close maintenance → vehicle Available (unless Retired)	On maintenance close, check current status isn't Retired before reverting

Build these as DB transactions, not just app-level updates — this is what makes the demo bulletproof when you deliberately try to break it live.

3. API Endpoints (Express)

```
POST    /auth/login
POST    /auth/signup  (fleet_manager only, self-elevation blocked)

GET      /vehicles           ?status=&type=&region=
POST     /vehicles
PATCH   /vehicles/:id

GET      /drivers           ?status=
POST     /drivers
PATCH   /drivers/:id

GET      /trips
POST     /trips             (create as Draft)
PATCH   /trips/:id/dispatch (validates + cascades status)
PATCH   /trips/:id/complete (captures odometer/fuel, cascades status)
PATCH   /trips/:id/cancel  (cascades status)

POST     /maintenance
PATCH   /maintenance/:id/close

POST     /fuel-logs
POST     /expenses
```

```

GET    /dashboard/kpis
GET    /reports/fuel-efficiency
GET    /reports/utilization
GET    /reports/cost
GET    /reports/roi
GET    /reports/export?format=csv

POST   /ai/dispatch-suggest      (wow feature)
POST   /ai/cost-forecast        (wow feature)
GET    /ai/fleet-insights       (wow feature)

```

4. Screens (mapped 1:1 to requirements doc)

1. **Login/Signup** — RBAC, session validation
 2. **Dashboard** — KPI cards (Active Vehicles, Available, In Maintenance, Active Trips, Pending, Drivers On Duty, Fleet Utilization %), filters by type/status/region
 3. **Vehicle Registry** — CRUD + search/filter/sort
 4. **Driver Management** — CRUD, license expiry visibility
 5. **Trip Management** — create/dispatch/complete/cancel with live validation errors
 6. **Maintenance** — log creation, auto status cascade
 7. **Fuel & Expense** — log entry, auto cost rollup per vehicle
 8. **Reports & Analytics** — charts + CSV export
-

5. Wow Features (Claude/OpenAI API) — pick 2-3, don't do all 4

☐ Priority 1: AI Dispatch Advisor

On trip creation, call the API with available vehicles/drivers + trip requirements (cargo weight, distance) → returns ranked recommendation with a one-line reason ("Van-05 — best capacity fit, driver has highest safety score on this route type"). This is your single strongest live-demo moment: type in a trip, watch a smart suggestion appear, judges immediately understand it's not just CRUD.

Prompt sketch:

```

Given available vehicles: [...], available drivers: [...],
trip: cargo_weight=X, distance=Y —
Rank the top match and give a one-sentence reason. Return JSON: {vehicle_id,
driver_id, reason}

```

☐ Priority 2: AI Fleet Insights (Natural-language query bar)

A single search box on the Reports screen: "Which vehicles are costing the most this month?" → Claude reads your aggregated JSON (not raw DB access — just pass it summarized stats) and answers in plain English. Cheap to build (no DB write risk), high wow factor because it looks conversational.

☐ Priority 3 (only if time remains): License Risk Summary

On the Safety Officer dashboard, send driver list + expiry dates to the API, get back a short prioritized risk summary ("3 drivers expire within 14 days — Alex, Priya, Raj"). Nice but lowest priority since it's mostly a formatting/sorting problem you could also solve without AI.

Important: for all three, keep it a *thin* wrapper — pass Claude clean structured data (JSON), ask for JSON back, render it. Don't let the model touch the database directly; that's slower and riskier to debug live.

6. Hour-by-Hour Plan (2 people, 8 hours)

Time	Person A (Backend)	Person B (Frontend)
0:00–0:30	Schema + migrations, project scaffold	Project scaffold, routing, login/signup UI
0:30–1:30	Auth + RBAC middleware	Dashboard shell, KPI cards (static data)
1:30–2:30	Vehicle + Driver CRUD APIs	Vehicle Registry + Driver Management screens, wire to API
2:30–4:00	Trip lifecycle + atomic status-cascade logic (the core rules)	Trip Management screen, dispatch/complete/cancel UI + error states
4:00–4:30	Lunch/break, quick sync	
4:30–5:15	Maintenance + Fuel/Expense APIs	Maintenance + Fuel/Expense screens
5:15–6:00	Reports aggregation queries (utilization, cost, ROI)	Reports screen + charts (recharts)
6:00–7:00	AI Dispatch Advisor + AI Insights endpoint	Wire AI features into Trip + Reports screens
7:00–7:45	Bug bash together — deliberately try to break every rule	Same — polish states, loading, empty states
7:45–8:00	Rehearse demo script (below)	Same

7. Demo Script (rehearse this exact sequence)

1. Login as Fleet Manager → show dashboard KPIs
2. Register vehicle "Van-05" (500kg capacity)
3. Register driver "Alex"
4. Create trip, cargo 450kg → **AI Dispatch Advisor suggests Van-05 + Alex with reason**

5. Dispatch → show vehicle/driver flip to "On Trip" live
6. Try to dispatch same vehicle again elsewhere → show it's blocked, explain the rule out loud
7. Complete trip → both flip back to Available, enter fuel consumed
8. Create maintenance log on same vehicle → show it vanish from dispatch pool instantly
9. Go to Reports → ask the **AI insights bar** a natural-language question, get a live answer
10. Close on the ROI/cost chart updating in real time

This script hits every mandatory rule *and* both AI wow features in under 3 minutes — that's your winning move.

8. Things to explicitly skip (don't let scope creep in)

- Dark mode (bonus only, do last if time allows)
- Email reminders (mock as an in-app notification instead — much faster, same demo value)
- PDF export (optional per doc — CSV is mandatory, do that only)
- Vehicle document management (bonus, skip)